

🕒 CHRONOS ### Automated Multi-Source Forensic Timeline Parser & Incident Reconstruction Engine [![Python](https://img.shields.io/badge/Python-3.10%2B-blue.svg?style=for-the-badge&logo=python&logoColor=white)](https://www.python.org/) [![React](https://img.shields.io/badge/React-19-61DAFB.svg?style=for-the-badge&logo=react&logoColor=black)](https://react.dev/) [![Vite](https://img.shields.io/badge/Vite-6.0-646CFF.svg?style=for-the-badge&logo=vite&logoColor=white)](https://vitejs.dev/) [![TypeScript](https://img.shields.io/badge/TypeScript-5.0-3178C6.svg?style=for-the-badge&logo=typescript&logoColor=white)](https://www.typescriptlang.org/) [![MITRE ATT&CK](https://img.shields.io/badge/MITRE%20ATT%26CK-v14-red.svg?style=for-the-badge)](https://attack.mitre.org/) [![Tests](https://img.shields.io/badge/Tests-20%20Passing-brightgreen.svg?style=for-the-badge)](https://github.com/BaggaByte/chronos)

Reconstruct complex multi-stage cyber breaches from raw, heterogeneous logs into defensible, chronologically accurate attacker narratives.

[Key Features](#-key-features) • [Architecture](#-architecture) • [Quick Start](#-quick-start) • [Investigation Dashboard](#-interactive-investigation-dashboard) • [Verification & Tests](#-verification--tests) ---

🔪 Problem Statement & Mission

Following a data breach or exfiltration incident, incident responders and digital forensics teams are tasked with reconciling hundreds of thousands of disparate log events across heterogeneous systems:

- **Disparate Log Formats:** Linux `auth.log`, Windows `EVTX`, Apache/Nginx web access logs, Cisco firewall syslogs, and AWS CloudTrail.
- **Timestamp & Timezone Chaos:** Unsynchronized system clocks, missing timezone offsets, syslog timestamps lacking year declarations, and daylight saving time (DST) shifts.
- **Correlation Paralysis:** Connecting an initial web exploit to credential theft on a domain controller, lateral movement via SSH, and staging/exfiltration into cloud storage.
- **Evidence Integrity Loss:** Lack of cryptographic chain of custody during log transformations and ingestion.

The Solution: Chronos

Chronos is an automated digital forensics and incident response (DFIR) platform that:

1. Cryptographically seals and streams heterogeneous log files (SHA-256 chain-of-custody).
2. Automatically normalizes all timestamps to universal UTC, explicitly flagging timezone inferences and clock drifts.
3. Tags MITRE ATT&CK tactics and techniques using rule-driven signature engines.
4. Correlates cross-host, multi-system events into unified **Attacker Narrative Threads**.
5. Flags statistical anomalies, execution volume spikes (z-score), and suspicious dwell-time gaps.
6. Presents findings in an **interactive, zoomable chronological dashboard** with exportable court- and executive-ready forensic reports.

⚡ Key Features

1. Multi-Format Streaming Ingestion

- Native streaming parsers for:
- **Linux Syslog** / `auth.log`: Password/pubkey authentication, sudo elevation, user disconnects.

- **Windows EVTX / Security Events:** Process creation (Event 4688), logon events (Event 4624), privilege escalation (Event 4672).
- **Web Server Logs (Apache/Nginx Common & Combined):** Path discovery, web exploitation (`POST /login` , command injections).
- **Network / Cisco ASA Syslog:** Teardown/built connections, external and internal IP mapping.
- **Cloud Audit Logs (AWS CloudTrail):** S3 `PutObject` exfiltration, IAM enumeration, AssumeRole calls.
- **Evidentiary Integrity:** Raw files are hashed (SHA-256) upon first intake; an immutable `ingest_manifest.json` tracks source hashes, file sizes, and record counts.

2. Universal UTC Normalization Engine

- Automatically parses RFC 3339 / ISO 8601, Unix epoch timestamps, Apache date-time format, and legacy BSD syslog headers.
- Infers missing timezone offsets based on declared host metadata while flagging them as `offset_inferred: true` for analyst defensibility.
- Identifies and tags un-synchronized host clocks (e.g., +47s drift).

3. MITRE ATT&CK Categorization & Tagging

- Tagging engine maps raw forensic events to explicit ATT&CK techniques:
- **Initial Access:** `T1190` (Exploit Public-Facing Application), `T1566` (Phishing).
- **Credential Access:** `T1003` (OS Credential Dumping / Mimikatz activity).
- **Privilege Escalation:** `T1548` (Abuse Elevation Control / sudo abuse).
- **Lateral Movement:** `T1021.004` (SSH Lateral Movement), `T1078` (Valid Accounts).
- **Collection & Exfiltration:** `T1560` (Archive Collected Data via `tar`), `T1041` (Exfiltration Over C2/Cloud).

4. Cross-Host Attacker Narrative Correlation

- Correlates multi-system evidence across four distinct entity dimensions: **IP addresses**, **User accounts**, **Session/Process IDs**, and **Target Hostnames**.
- Separates benign background noise (e.g., automated cron jobs, routine logins) from malicious activity, extracting isolated attacker threads.

5. Statistical Anomaly & Time-Gap Detection

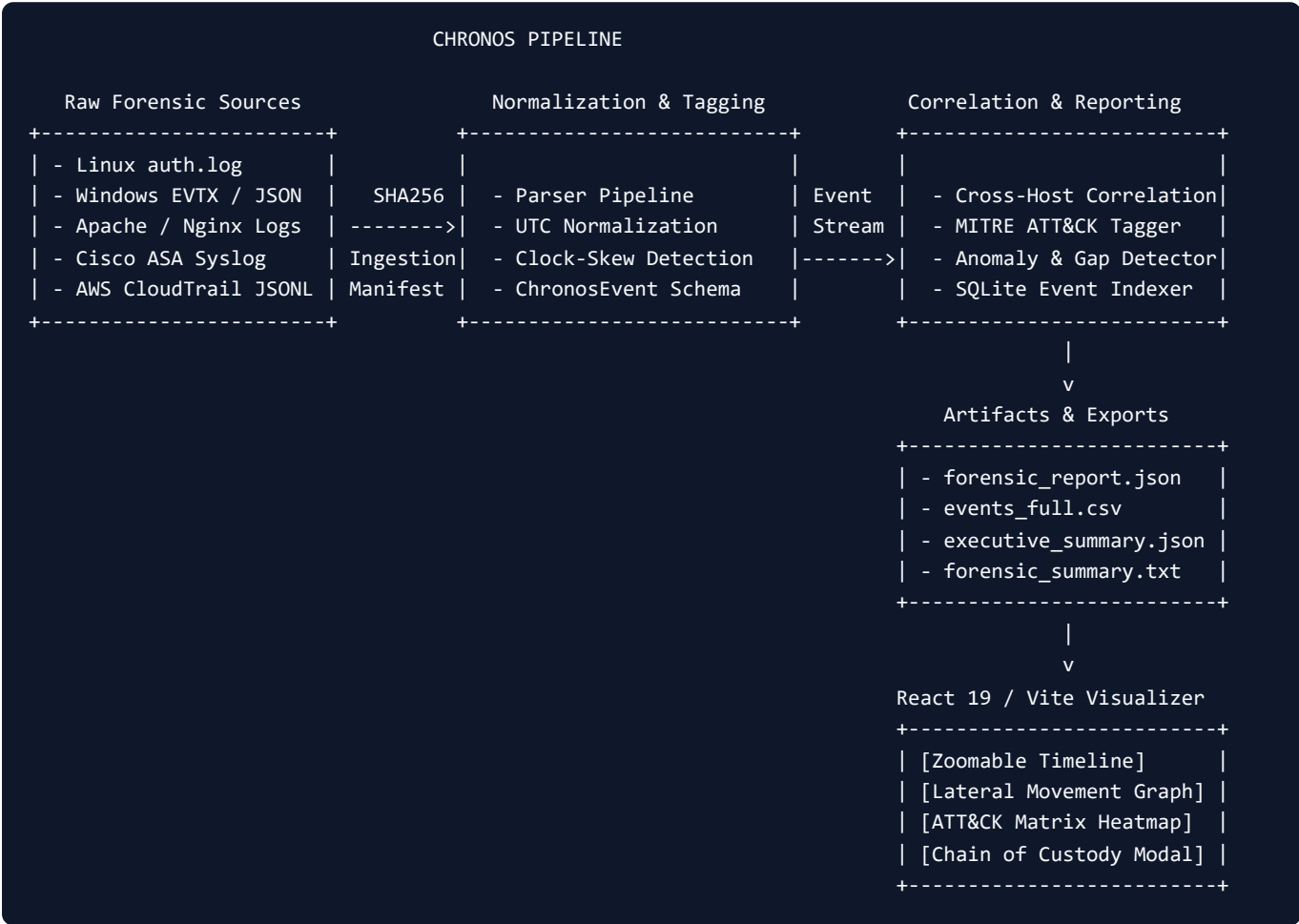
- **Rolling Z-Score Spike Detection:** Identifies concentrated brute-force attacks and authentication storms ($Z > 2.5$).
- **Adaptive Dwell-Time Gap Detection:** Highlights anomalous pauses between attacker stages, pinpointing manual reconnaissance vs. automated scripting.

6. Modern Interactive React UI

- **Zoom & Pan Chronological Timeline:** Smooth mouse-wheel zooming and pointer drag-panning across dense log clusters.
- **Interactive MITRE ATT&CK Matrix:** Technique heatmaps indicating incident coverage.

- **Cross-Host Lateral Movement Graph:** Visual node-link diagram mapping breach progression from internet edge to cloud exfiltration.
- **Statistical Anomalies & Spike Inspector:** Visual breakdown of flagged execution spikes and time gaps.
- **Forensic Report Export Hub:** One-click download of comprehensive forensic packages (JSON, CSV, and formatted forensic summaries).

Architecture



Quick Start

1. Prerequisites

- Python 3.10+
- Node.js 18+ &

npm

2. Clone the Repository

```
git clone https://github.com/BaggaByte/chronos.git
cd chronos
```

3. Backend Pipeline (Python)

Install Python dependencies:

```
pip install -r requirements.txt
```

Generate the multi-stage aerospace attack dataset:

```
python scripts/generate_synthetic_dataset.py
```

Run the complete Chronos ingestion, correlation, and anomaly detection pipeline:

```
python scripts/run_pipeline.py
```

Outputs generated:

- `data/synthetic/` : Raw heterogeneous logs + ground truth
- `data/chronos_events.db` : Fast, time-indexed SQLite event store
- `data/forensic_report.json` : Full correlation narrative and detected anomalies
- `data/exports/` : Ready-to-use CSV, JSON, and forensic summary reports
- `ui/src/data/` : Auto-synchronized datasets for the React visualizer

4. Interactive Visualizer (Frontend)

Navigate to the `ui/` directory:

```
cd ui
npm install
npm run dev
```

Open your browser at:

```
http://localhost:5173
```

Interactive Investigation Dashboard

View	Description
Incident Narrative	Structured chronological walkthrough of the attacker's journey from initial access to data exfiltration.
Zoomable Timeline	Interactive SVG timeline with mouse-wheel zoom and drag-to-pan, color-coded by MITRE ATT&CK tactic.
Lateral Movement Graph	Node-link topological visualization tracking how the adversary traversed internal hosts (<code>web-ext-01</code> → <code>win-eng-07</code> → <code>lin-db-03</code> → <code>s3-aerospace-vault</code>).
ATT&CK Matrix	Tactic-by-tactic breakdown of detected techniques with occurrence frequencies.
Anomaly Inspector	Visualizes execution spikes (z-score analysis) and dwell-time gaps between incident phases.
Search & Filter Table	Real-time full-text search, filterable by host, severity, ATT&CK tag, and source file.
Chain of Custody	Cryptographic audit modal displaying SHA-256 hashes and evidentiary metadata.

Verification & Tests

Chronos includes an automated test suite verifying all parsing, UTC normalization, clock-skew, and multi-host correlation logic:

```
# Run all tests
python -m unittest discover tests

# Or run individual modules
python tests/test_parsers_and_norm.py
python tests/test_correlation.py
python tests/test_e2e_pipeline.py
```

Test Suite Coverage:

- `test_parsers_and_norm.py` : Validates all 5 log formats, time offset inference, missing year handling, and clock-skew flags.
- `test_correlation.py` : Confirms multi-hop entity joining correctly isolates attacker groups from benign background activity.
- `test_e2e_pipeline.py` : Validates end-to-end processing, ATT&CK technique tagging accuracy, and export generation.

Project Structure

```
chronos/
├── config/
│   └── attack_mappings.yaml           # MITRE ATT&CK rule definitions
├── data/
│   ├── synthetic/                   # Generated multi-source log files
│   ├── exports/                     # CSV, JSON, and TXT forensic reports
│   └── forensic_report.json          # Structured pipeline output
├── scripts/
│   ├── generate_synthetic_dataset.py # Synthesizes realistic 5-source attack logs
│   ├── generate_scenario_b.py        # Alternate incident scenario generator
│   └── run_pipeline.py               # Main pipeline runner
├── src/
│   ├── anomaly/                     # Z-score spike detector & dwell-time gap engine
│   ├── correlation/                 # Multi-hop cross-host correlation engine
│   ├── ingestion/                   # Cryptographic SHA-256 ingestion & manifest
│   ├── normalization/              # Universal UTC normalization & clock-skew engine
│   ├── parsers/                     # AuthLog, EVTIX, Apache, Cisco, and CloudTrail parsers
│   ├── reporting/                   # Multi-format report exporter (CSV/JSON/TXT)
│   ├── storage/                     # SQLite time-indexed persistence layer
│   └── schema.py                    # ChronosEvent and CorrelationGroup dataclasses
├── tests/                           # Comprehensive test suite (20 unit/E2E tests)
├── ui/                              # Modern React 19 + Vite + Tailwind CSS dashboard
│   ├── src/
│   │   ├── components/chronos/      # Timeline, Matrix, Host Graph, Narrative, etc.
│   │   └── data/                     # Live forensic data feeds
│   ├── package.json
│   └── vite.config.ts
├── implementation-plan-final.md      # Architectural specification
├── requirements.txt                  # Python dependencies
└── README.md                         # Documentation
```

License

Built for the Forensics & Incident Response (PS-10) Challenge. Distributed under the MIT License.